

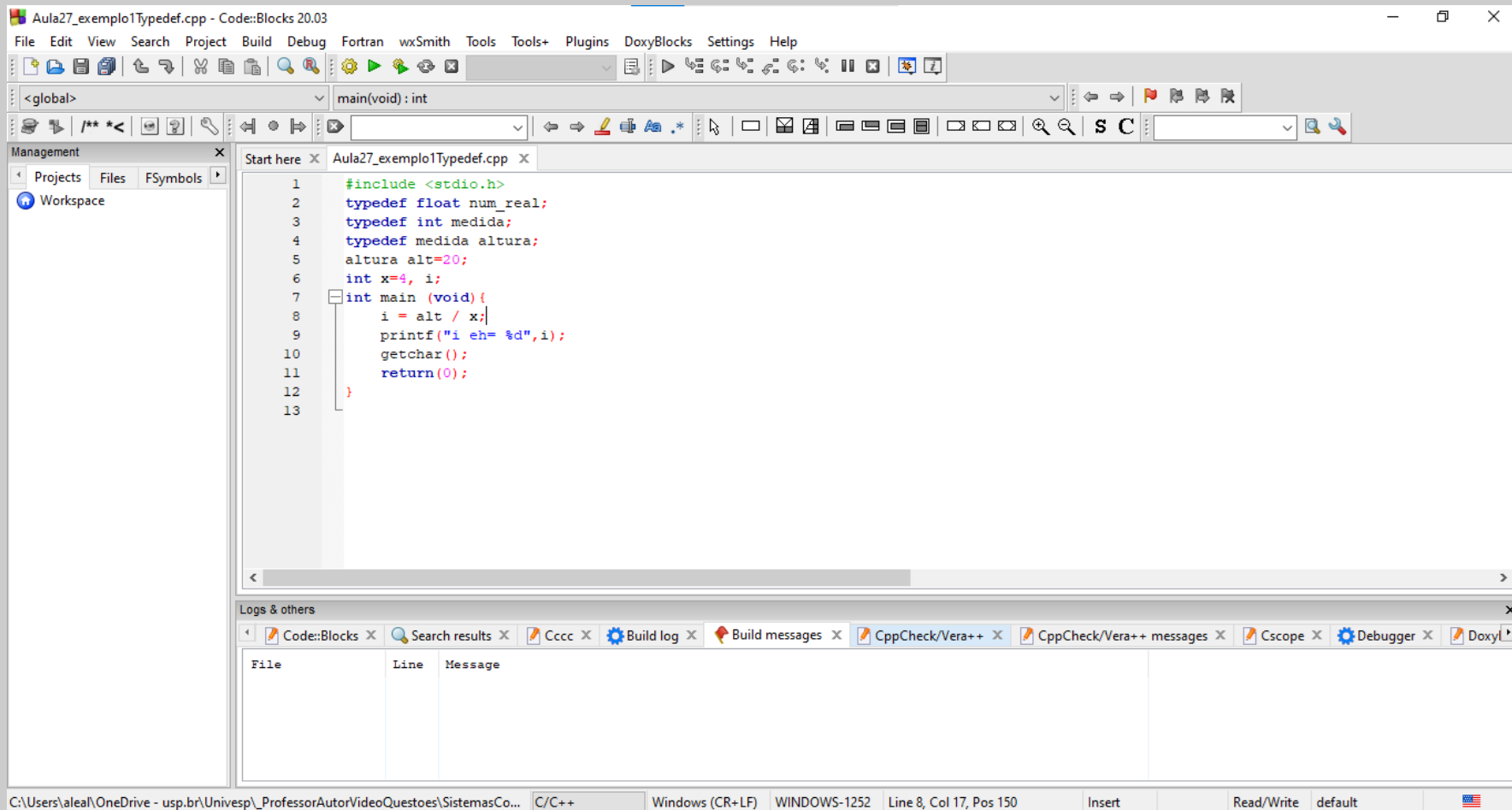
SISTEMAS COMPUTACIONAIS

**Linguagem C: simulação de
paginação e particionamento
fixo**

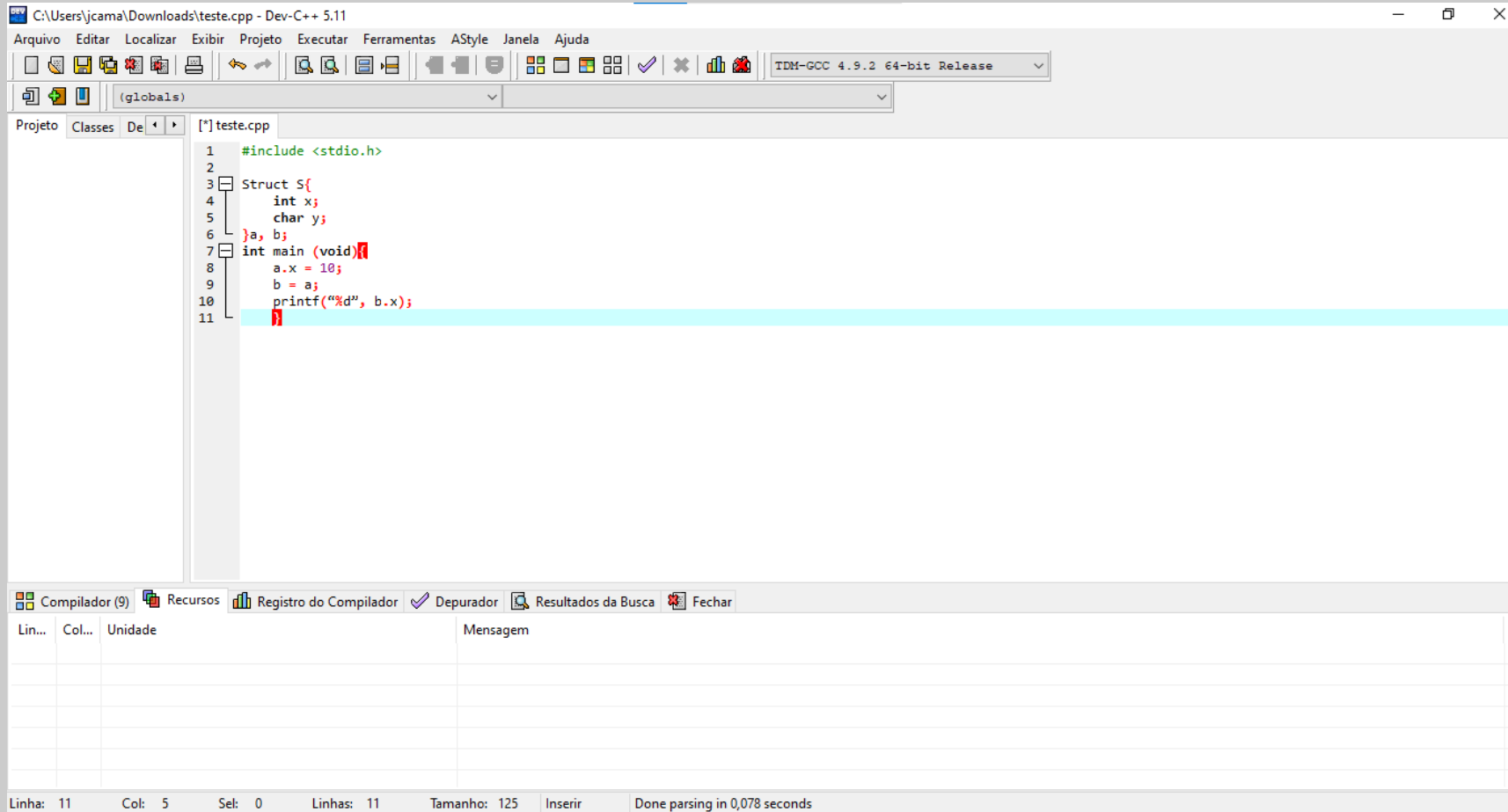
PREPARANDO

- **Baixar software para desenvolvimento de aplicações em C**
 - **Visual Studio Code:**
<https://code.visualstudio.com//>
 - **Dev-C**
<https://sourceforge.net/projects/orwellddevcpp>
 - **Code::Blocks www.codeblocks.org**

CodeBlocks



DevC



The screenshot shows the Dev-C++ 5.11 IDE interface. The title bar indicates the file path: C:\Users\jcama\Downloads\teste.cpp - Dev-C++ 5.11. The menu bar includes Arquivo, Editar, Localizar, Exibir, Projeto, Executar, Ferramentas, AStyle, Janela, and Ajuda. The toolbar contains icons for file operations, compilation, and execution. The compiler is set to TDM-GCC 4.9.2 64-bit Release. The left sidebar shows the Project Explorer with a tree view for 'teste.cpp' containing a struct 'S' and a 'main' function. The main editor window displays the following C++ code:

```
1 #include <stdio.h>
2
3 Struct S{
4     int x;
5     char y;
6 }a, b;
7 int main (void){
8     a.x = 10;
9     b = a;
10    printf("%d", b.x);
11 }
```

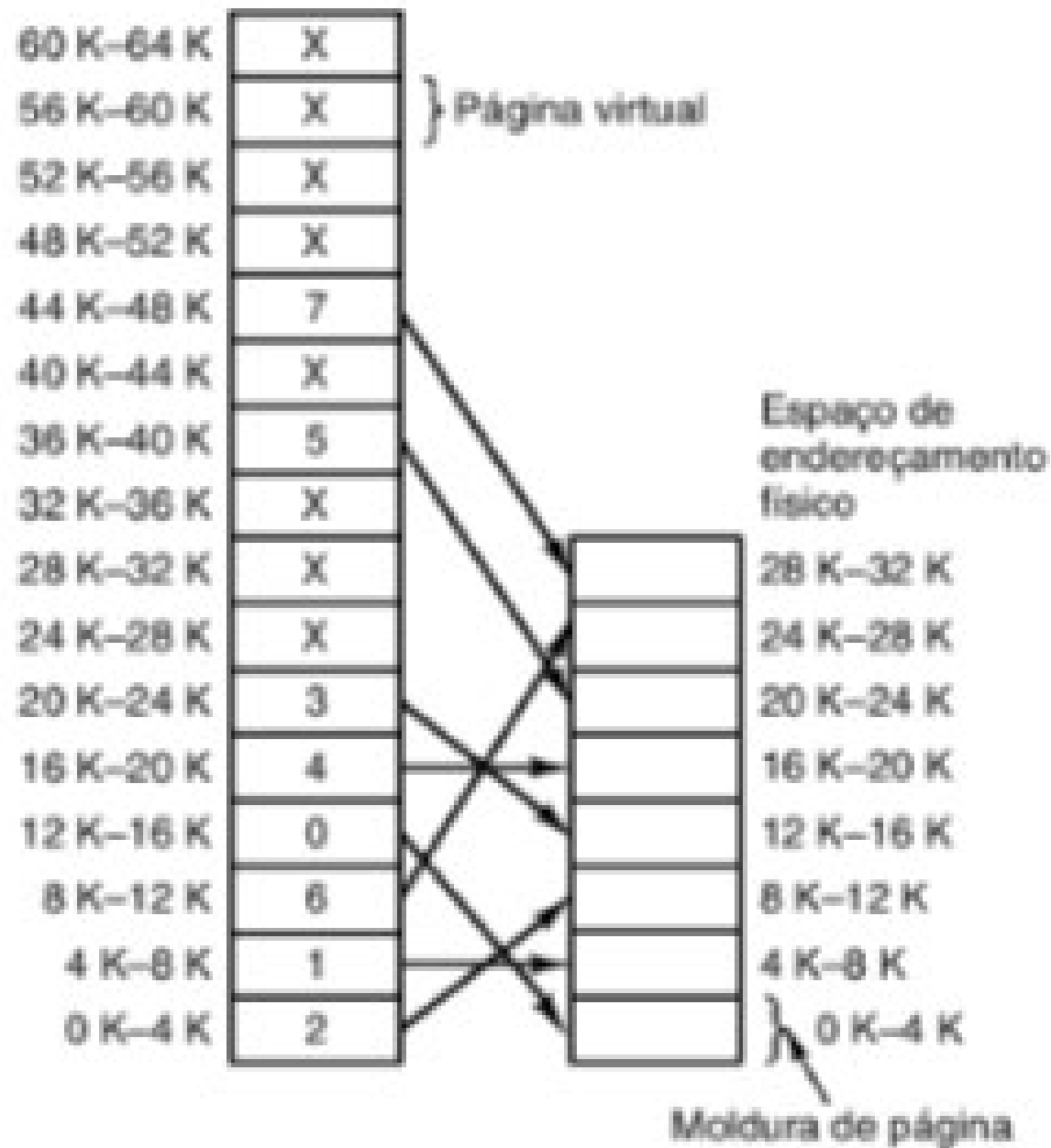
The bottom status bar shows the following information: Linha: 11, Col: 5, Sel: 0, Linhas: 11, Tamanho: 125, Inserir, Done parsing in 0,078 seconds. The bottom panel includes tabs for Compilador (9), Recursos, Registro do Compilador, Depurador, Resultados da Busca, and Fechar. The bottom-most section contains a table with columns Lin..., Col..., and Unidade, and a large area for Mensagem.

PAGINAÇÃO

- Técnica de memória virtual e de quebra de processos em páginas que são mapeadas em frames na memória principal
 - Páginas virtuais com endereçamento virtual
 - Páginas com mistura de tipos de dados
 - Páginas de tamanho fixo no disco
 - Frames (ou molduras) na memória principal
 - Tabela de páginas são estruturas para mapear uma página ao frame correspondente
 - Cada processo tem a sua tabela de páginas com seu endereçamento virtual correspondente a um endereço físico onde a informação está armazenada



ESTRUTURAS DE PAGINAÇÃO



ESTRUTURA DE PAGINAÇÃO

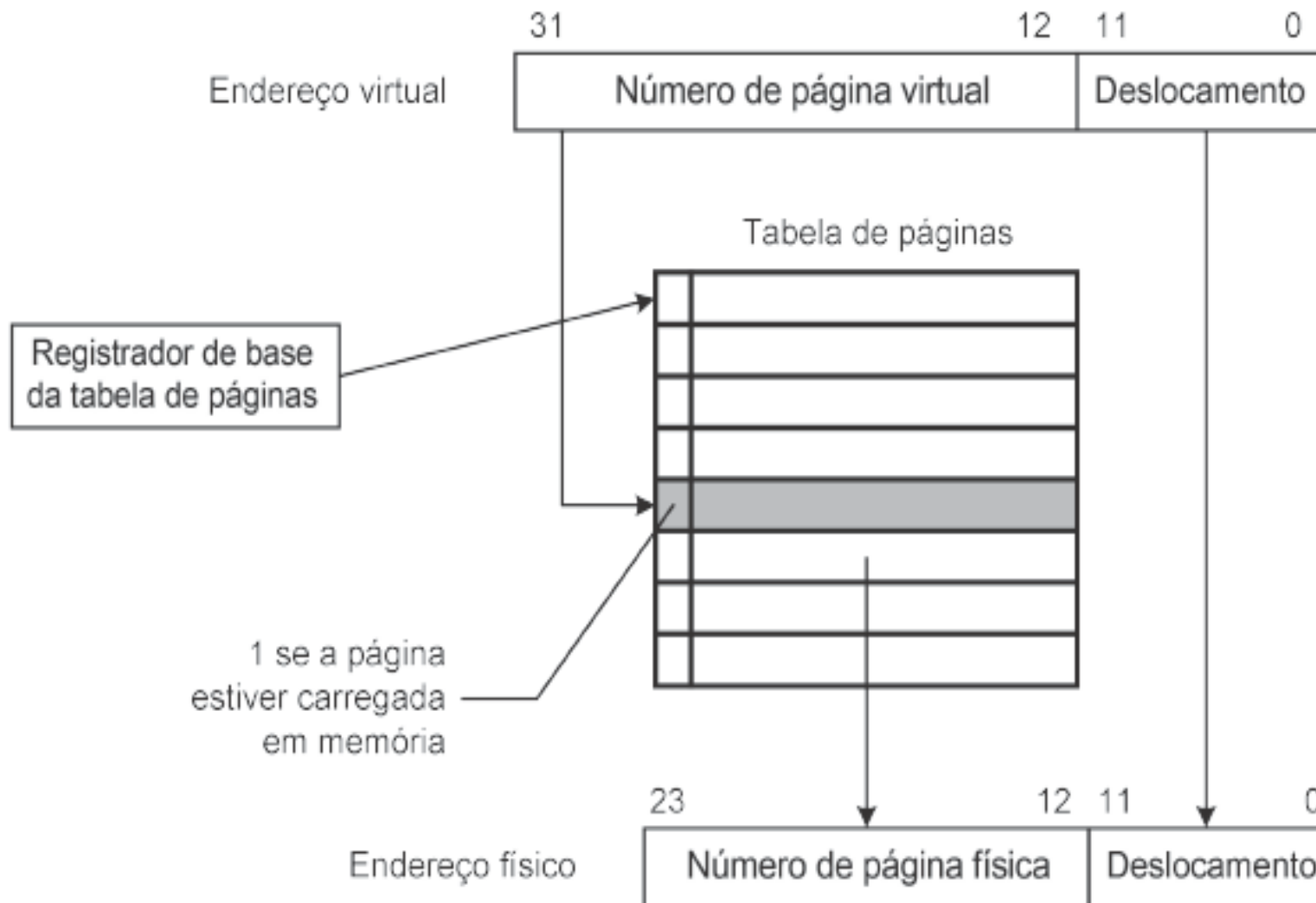


Fig. 7.26 – Tradução de endereços em uma memória virtual paginada

ELEMENTOS: ENDEREÇOS

- No DISCO cada página tem:
 - Bit de validade (bv=1 pv na RAM)
 - Ponteiro para página virtual (pv)
 - Deslocamento (dv)
- Na RAM cada frame tem:
 - Número da página física (pf)
 - Deslocamento (df)
 - Conteúdo



ESTRUTURAS EM C: RAM E DISCO

```
typedef struct moldura{  
    int bv;  
    char contentRAM;  
} frame;  
typedef struct pagina{  
    int bv;  
    char contentDISCO;  
    frame *pv;  
}page;
```

```
frame RAM[2];  
page DISCO[20];
```

C:\Users\aleal\OneDrive - usp.br\Univesp\ProfessoraVideoQuestoes\SistemasComputacionais1Bim2021\Semana5\paginacao.cpp - [Ex]

Arquivo Editar Localizar Exibir Projeto Executar Ferramentas AStyle Janela Ajuda

```
[*] paginacao.cpp
8 typedef struct pagina{
9     int bv;
10    char contentDISCO;
11    frame *pv;
12 }page;
13 frame RAM[2];
14 page DISCO[20];
15 int i,j;
16 int main() {
17     //Iniciando da RAM
18     for (i=0;i<2; i++)
19         RAM[i].bv=0;
20     //paginacao
21     for (i=0; i<20; i++) {
22         DISCO[i].contentDISCO='a'+i;
23         DISCO[i].bv=1;
24         DISCO[i].pv=NULL;
25         //simulacao que a 5a e a 10a paginas sao mais usadas entao sobe na RAM
26         if ((DISCO[i].contentDISCO == 'e') || (DISCO[i].contentDISCO == 'j'))
27             for (j=0;j<2;j++) // aloca mais usado no 1o espaco da RAM disponivel
28                 if (RAM[j].bv==0) {
29                     RAM[j].bv=1;
30                     DISCO[i].pv=&RAM[j]; //ponteiro do DISCO tem endereço da RAM que guarda a pagina
31                     RAM[j].contentRAM=DISCO[i].contentDISCO; j=2;
32                 }
33     }
34     printf("visualizacao DISCO\n");
35     for (i=0; i<20; i++) {
36         printf("%d page [content:%c",i+1,DISCO[i].contentDISCO);
37         printf(" BV:%d",DISCO[i].bv);
38         printf(" RAM:%d\n",DISCO[i].pv);
39     }
40     printf("\nvisualizacao RAM\n");
41     for (j=0; j<2; j++) {
42         printf("%d frame [content:%c",j+1,RAM[j].contentRAM);
43         printf(" BV:%d\n",RAM[j].bv);
44     }
45     return(0);
46 }
```

Compilador Recursos Registro do Compilador Depurador Resultados da Busca

C:\Users\aleal\OneDrive - usp...

```
visualizacao DISCO
1 page [content:a BV:1 RAM:0]
2 page [content:b BV:1 RAM:0]
3 page [content:c BV:1 RAM:0]
4 page [content:d BV:1 RAM:0]
5 page [content:e BV:1 RAM:4223040]
6 page [content:f BV:1 RAM:0]
7 page [content:g BV:1 RAM:0]
8 page [content:h BV:1 RAM:0]
9 page [content:i BV:1 RAM:0]
10 page [content:j BV:1 RAM:4223048]
11 page [content:k BV:1 RAM:0]
12 page [content:l BV:1 RAM:0]
13 page [content:m BV:1 RAM:0]
14 page [content:n BV:1 RAM:0]
15 page [content:o BV:1 RAM:0]
16 page [content:p BV:1 RAM:0]
17 page [content:q BV:1 RAM:0]
18 page [content:r BV:1 RAM:0]
19 page [content:s BV:1 RAM:0]
20 page [content:t BV:1 RAM:0]
```

```
visualizacao RAM
1 frame [content:e BV:1]
2 frame [content:j BV:1]

-----
Process exited after 1.904 seconds with
return value 0
Pressione qualquer tecla para continuar
...
```

MULTIPROGRAMAÇÃO

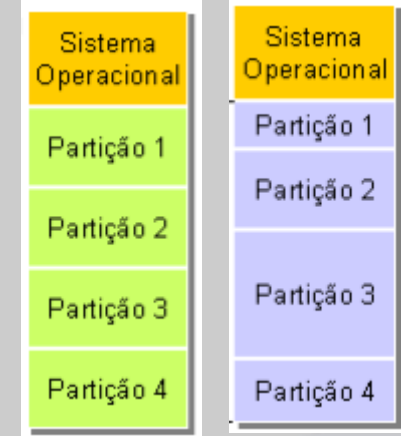
- Contexto: hierarquia de memórias

- Monoprogramação
- Multiprogramação

A- Particionamento fixo com tamanho igual

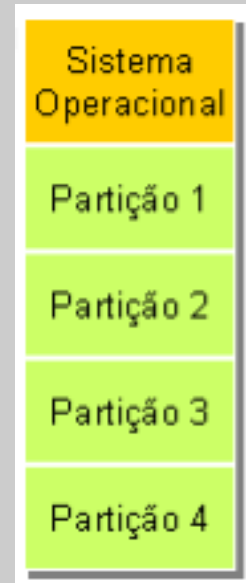
B- Particionamento fixo com tamanho variável

C- Particionamento variável ou dinâmico



PARTICIONAMENTO FIXO COM TAMANHO FIXO

```
typedef struct {  
    unsigned int bv; // 0 = partição vazia  
    char processo[5];  
} particao; particao RAM[10];  
  
int main() {  
    inicialização da RAM  
    alocação processo na RAM  
    execução  
}
```



```
C:\Users\aleal\OneDrive - usp.br\Univesp\ProfessoraVideoQuestoes\SistemasComputacionais1Bim2021
Arquivo  Editar  Localizar  Exibir  Projeto  Executar  Ferramentas  AStyle  Janela  Ajuda

paginacao.cpp  particionamento.cpp

9  int main() {
10  int i;
11  //inicializacao
12  for (i=0; i<10; i++)
13      RAM[i].bv=0;
14
15  //alocacao. Será ideal transformar em função
16  for (i=0; i<10; i++)
17      if (RAM[i].bv == 0) {
18          printf("Digite um processo: ");
19          RAM[i].bv = 1;
20          scanf("%s",&RAM[i].processo);
21      }
22
23  //simulacao do processador encerrando execucao de processo
24  strcpy(RAM[2].processo,"");RAM[2].bv=0; printf("\n3o Processo finalizado\n");
25  strcpy(RAM[8].processo,"");RAM[8].bv=0; printf("\n9o Processo finalizado\n");
26
27  //visualizacao da RAM simulada. Será ideal transformar em funcao
28  printf("\nStatus da RAM\n");
29  for (i=0; i<10; i++)
30      printf("%da Particao da RAM: [%d|%s]\n",i+1,RAM[i].bv,RAM[i].processo);
31
32  //NOVA alocao
33  for (i=0; i<10; i++)
34      if (RAM[i].bv == 0) {
35          printf("\n");
36          printf("Digite um processo: ");
37          RAM[i].bv = 1;
38          scanf("%s",&RAM[i].processo);
39      }
40
41  //NOVA visualizacao da RAM simulada
42  printf("\nStatus da RAM\n");
43  for (i=0; i<10; i++)
44      printf("%da Particao da RAM: [%d|%s]\n",i+1,RAM[i].bv,RAM[i].processo);
45  return(0);
46  }
```

```
C:\Users\aleal\OneDrive...
Digite um processo: do
Digite um processo: while
Digite um processo: switch
Digite um processo: case
Digite um processo: if
Digite um processo: else
Digite um processo: for
Digite um processo: printf
Digite um processo: scanf
Digite um processo: int

3o Processo finalizado
9o Processo finalizado

Status da RAM
1a Particao da RAM: [1|do]
2a Particao da RAM: [1|while]
3a Particao da RAM: [0|]
4a Particao da RAM: [1|case]
5a Particao da RAM: [1|if]
6a Particao da RAM: [1|else]
7a Particao da RAM: [1|for]
8a Particao da RAM: [1|printf]
9a Particao da RAM: [0|]
10a Particao da RAM: [1|int]

Digite um processo: SWITCH
Digite um processo: SCANF

Status da RAM
1a Particao da RAM: [1|do]
2a Particao da RAM: [1|while]
3a Particao da RAM: [1|SWITCH]
4a Particao da RAM: [1|case]
5a Particao da RAM: [1|if]
6a Particao da RAM: [1|else]
7a Particao da RAM: [1|for]
8a Particao da RAM: [1|printf]
9a Particao da RAM: [1|SCANF]
10a Particao da RAM: [1|int]
```

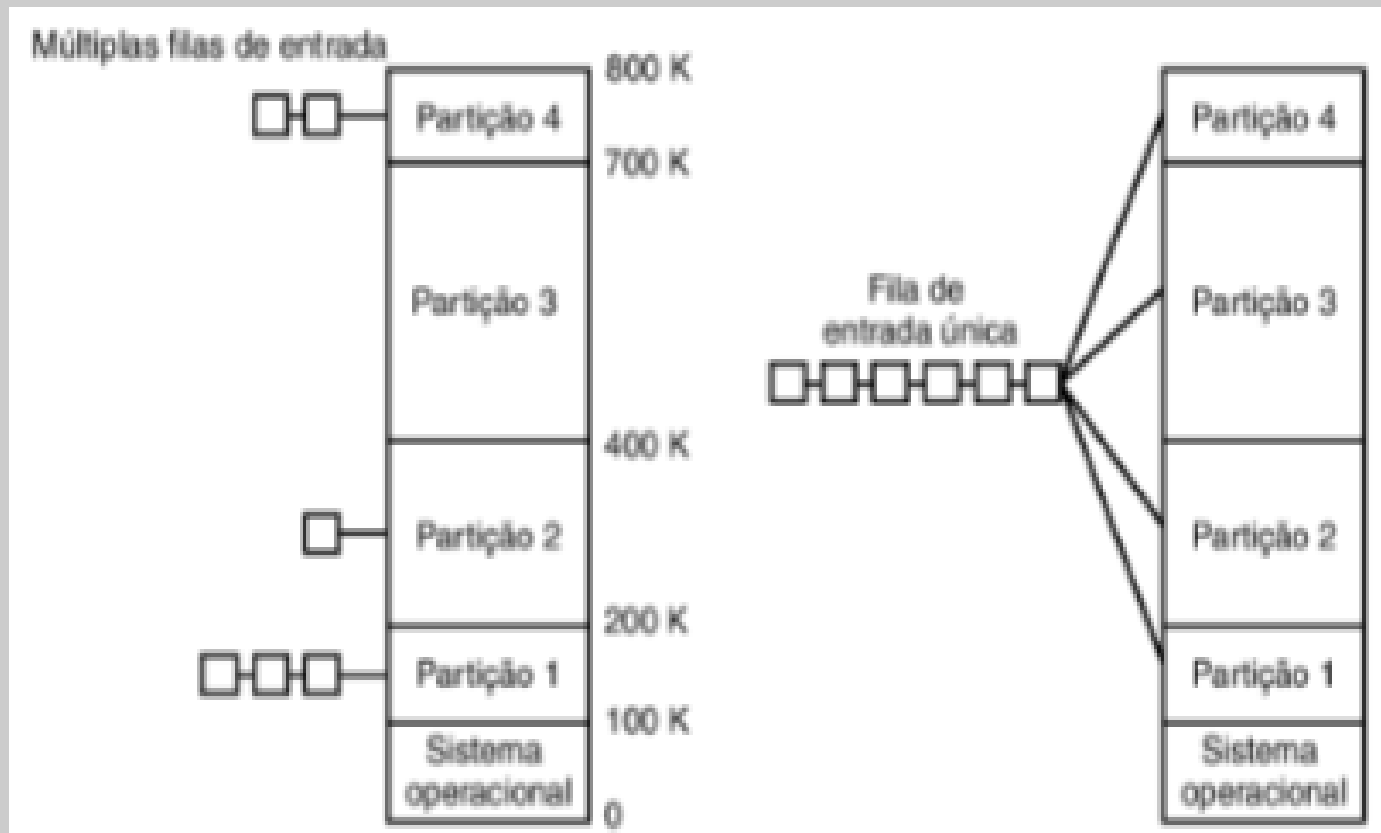
Compilador Recursos Registro do Compilador Depurador Resultados da Busca

Linha: 35 Col: 18 Sel: 0 Linhas: 46 Tamanho: 1250 Inserir Done parsing in 0,016 seconds

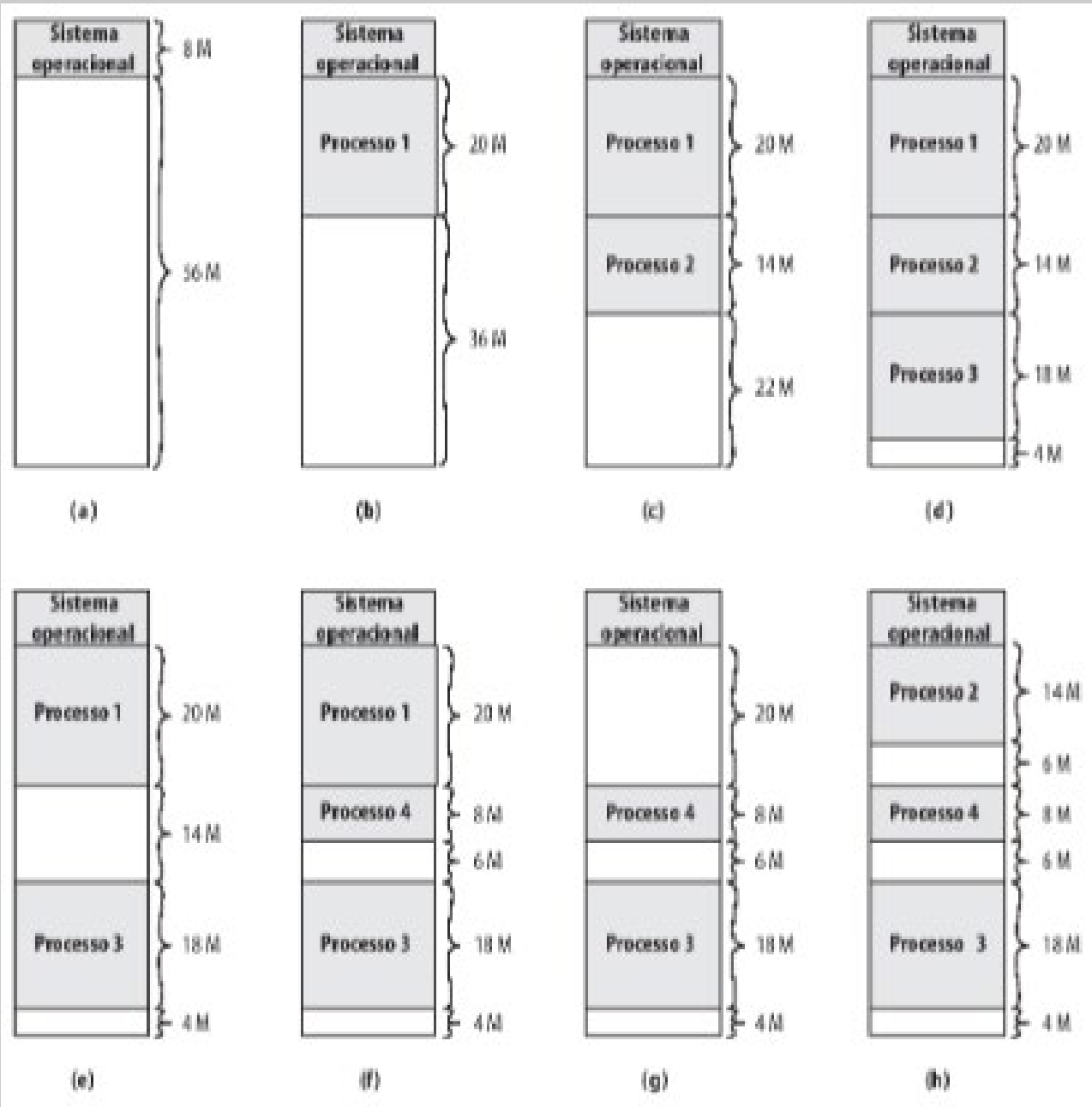
Otimização:

Função
- inicialização
- alocação

PARTICIONAMENTO FIXO DINÂMICO



PARTICIONAMENTO VARIÁVEL E DINÂMICO



BIBLIOGRAFIA

- **SCHILDT, H., C Completo e Total, Makron Books, 1997. – Capítulo 2.**
- **KELLEY, A.; POHL, I., A Book on C, Addison-Wesley, quarta edição, 1998. – Capítulos 1, 2 e 4.**
- **Damas, L. Linguagem C. Editora Grupo GEN. 10/2006. VBID 9788521632474**
- **Soffner, R. Algoritmos e Programação em Linguagem C, 1ª edição. Editora Saraiva. 08/2013. VBID 9788502207530**

SISTEMAS COMPUTACIONAIS

**Linguagem C: simulação de
paginação e particionamento
fixo**